STANFORD CS336 · LANGUAGE MODELING FROM SCRATCH

第 02 课复习资料

PyTorch 与资源核算

目录

1 先定位：从“怎样表示文本”走向“怎样核算资源” 2

2 接上主线：张量里到底存了什么，占多少空间？ 3

3 理解操作：用 einops 把形状与计算说清楚 5

4 核心例子：从一次矩阵乘法推到整次训练 6

5 理解速度：为什么算得少，也可能被显存带宽卡住？ 8

6 把账算完整：训练状态、优化器与两种省显存方法 10

7 一页速查：把本课压回一条主线 13

8 自测：合上前文，检查自己是否真的理解 14

9 参考答案与资料来源 15

阅读主线：文本变成张量之后，怎样判断模型能否装下、要算多久，以及究竟慢在哪里？

1 先定位：从“怎样表示文本”走向“怎样核算资源”

1.1 接上第一课：表示方案最终会变成资源开销

第一课把文本变成 token ID，随后由嵌入表变成向量。第二课继续追问：这些向量、模型参数及训练时的中间结果要占多少显存？运算需要多少计算？为什么 GPU 峰值算力很高，实际训练仍然可能很慢？

> **本课的核心问题：给定模型、数据和硬件，先算清“装不装得下”，再估计“算多久”，最后找出“算力还是搬运限制了速度”。**

PyTorch 是表达计算的工具，资源核算是贯穿全课的思维方式。只会写出正确结果，还不足以判断一个方案能否扩展到大模型。

1.2 三本账：容量、工作量与速度

问题	基本量与单位	核算方法
装不装得下？	存储量，byte	元素个数 × 每元素字节数；再汇总同时存活的张量。
总计算量？	运算量，FLOPs	从矩阵形状计数；把前向、反向及额外重算区分开。
耗时下界？	吞吐率、带宽	计算量除以 FLOP/s，搬运量除以 byte/s，判断哪一项更慢。

显存**容量**大，表示能装更多；显存**带宽**高，表示每秒能搬更多。两者不是同一指标。总 FLOPs 少，也不自动意味着实际耗时短。

1.3 连续例子：同一个小网络，逐步把账补完整

沿用官方讲义的等宽网络：输入 $X \in \mathbb{R}^{B\times d}$，每层权重 $W_\ell \in \mathbb{R}^{d\times d}$，依次执行

$$H_0 = X, \qquad H_\ell = \mathrm{ReLU}(H_{\ell-1}W_\ell), \qquad \ell = 1, \dots, L.$$

无偏置、各层不共享权重，参数量为 $N = Ld^2$。正文固定用 $B = 4, d = 8, L = 3$：共有 192 个参数，每层输出 32 个元素。后面依次为它补上形状、字节、FLOPs、梯度与优化器状态。

复习路线：第 2–6 节建立理解；第 7 节一页速查；第 8 节独立自测。正文依据正式与录像版讲义，来源和差异说明见第 9.2 节。

2 接上主线：张量里到底存了什么，占多少空间？

本节目标：看到 shape、dtype、device，就能开始核算；分清精度与范围。

2.1 先认清三个属性：形状、数据类型、位置

张量是多维数组；本课的 rank 表示**轴的数量**，不是线性代数中的矩阵秩。形状为 $(4, 8, 2)$ 的张量有 3 个轴、64 个元素。数据、参数、梯度、激活和优化器状态，都可用张量存储。

```
x = torch.zeros(4, 8)
x.shape          # (4, 8)
x.numel()        # 32
x.dtype          # torch.float32, under the usual default
x.element_size() # 4 bytes
x.device         # cpu, under the usual default
```

对普通稠密张量，逻辑数据量为

$$M_{\text{tensor}} = \left(\prod_i s_i\right) q, \qquad s_i = \text{第} i \text{ 个轴的长度}, \quad q = \text{每元素字节数}.$$

例如 $(4, 8)$ 的 fp32 数据为 $4 \times 8 \times 4 = 128$ byte；改为 bf16 则为 64 byte。多个张量若共享底层存储，不能把各自逻辑大小重复算作独立分配；实际显存还有分配器与临时缓冲区开销。

2.2 从小例子走到 Transformer 的形状

讲义展示 $(B, S, H, d_h) = (32, 16, 16, 64)$：批大小、序列长度、头数、每头维度。共有 524 288 个元素，fp32 占 2 MiB，bf16 占 1 MiB。

这里 d_h 是每头维度；后文 d 表示整个隐藏维度，勿混用。统一单位也很重要：1 GB $= 10^9$ byte，1 GiB $= 2^{30}$ byte。

2.3 为什么同样 16 bit，fp16 和 bf16 不一样？

浮点数可理解为符号、指数和有效数字三部分。**指数**主要决定能覆盖的数量级；**有效数字位数**主要决定同一数量级内分得多细。

类型	byte	指数 / 尾数	需要记住的取舍
fp32	4	8 / 23	范围大、分辨率较高，存储成本也高。
fp16	2	5 / 10	比 fp32 省空间，但范围较窄，易溢出或下溢。
bf16	2	8 / 7	指数范围接近 fp32，但相邻可表示数更疏。

表内为位数，三种类型另有 1 个符号位。讲义用 10^{-8} 演示：转为 fp16 会成为 0，bf16 仍非零；这**不意味着** bf16 具有 fp32 的精度，也不保证训练必然稳定。

2.4 混合精度：不同用途的张量，不必用同一种精度

大矩阵乘法希望省显存并利用低精度硬件；长期累积的优化器统计量则需要更稳健的表示。讲义用“参数、激活、梯度取 bf16，优化器状态取 fp32”做资源估算。**这是一个明确的核算配置，不是所有训练程序的默认布局。**

原始混合精度训练论文还讨论 fp32 主权重副本和 loss scaling；若实现保留主权重，必须额外计入显存。loss scaling 先放大损失以减少小梯度下溢，再在更新前还原梯度；它不能修复所有数值问题。

补充澄清：PyTorch 的 autocast 按运算选择计算类型；它不会把上下文中的一切张量永久转成 bf16，也不能据此断言参数、梯度和优化器状态都是低精度。

```
# Assuming CUDA is available, x and w are fp32 CUDA tensors.
with torch.amp.autocast("cuda", dtype=torch.bfloat16):
    y = x @ w   # eligible matmul may use bf16
    z = torch.zeros(4, 8, device="cuda")
    # z still uses the default dtype; autocast is not a global cast
```

做真实预算时，直接检查各类张量的 dtype，再乘元素数；不要从“开启 AMP”反推出固定的每参数字节数。

2.5 把张量放到 GPU：位置也是计算约定的一部分

CPU 内存和 GPU 显存是不同位置。普通运算的输入与权重需在兼容设备上；搬运数据也需要时间。

```
if torch.cuda.is_available():
    y = x.to("cuda:0")
    z = torch.zeros(32, 32, device="cuda:0")
```

`x.to(...)` 返回目标张量，需保存返回值。录像版还查询 GPU 数量、设备属性以及 `memory_allocated()`：两个独立的 $(32, 32)$ fp32 GPU 张量，逻辑数据合计 8192 byte。

2.6 再往下压位宽：fp8 与 fp4 的启示

讲义进一步介绍 FP8 的 E4M3、E5M2：前者分辨率较好、后者范围较大；按讲义采用的格式，最大有限值分别为 448 与 57344。它们需要结合缩放等数值策略，而非直接替换所有 fp32 数值。

NVFP4 把单个数值编码压到 4 bit，并引入分块缩放。**“4 bit”只描述数值编码本身**，总存储还包括缩放因子等信息；也依赖对应硬件与库支持。本课掌握动机和取舍即可，不要求实现低精度训练配方。

> **第一本账的原则：先列出实际存在的张量，再按它们各自的元素数与 dtype 计费；“模型用了低精度”不是完整的显存预算。**

3 理解操作：用 einops 把形状与计算说清楚

本节目标：能读懂 einsum、reduce、rearrange，并把形状接到后面的成本计数。

3.1 einsum：保留哪些轴，沿哪些轴求和？

下文的 `einsum`、`reduce`、`rearrange` 均从 `einops` 导入。对 $X \in \mathbb{R}^{B\times d}$、$W \in \mathbb{R}^{d\times k}$，线性变换为

$$Y_{bo} = \sum_{i=1}^{d} X_{bi} W_{io}, \qquad Y \in \mathbb{R}^{B\times k}.$$

```
y = einsum(x, w, "batch in, in out -> batch out")
```

出现在输出中的 `batch`,`out` 被保留；未出现在输出中的 `in` 被求和。轴名是形状说明，不会自动识别数据的真实语义；相同名称的轴仍须有正确对应关系。

讲义的成对相似度例子：$X : (B, S_1, d)$，$Y : (B, S_2, d)$。

```
z = einsum(x, y,
    "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
```

结果形状为 (B, S_1, S_2)，对每对位置沿 hidden 轴做内积。即使 $S_1 = S_2$，也应使用不同轴名，避免误写成同位置逐元素相乘。省略号 `...` 可表达共同的前导维度。

3.2 reduce：明确写出汇总方式

```
x = torch.ones(2, 3, 4)
y = reduce(x, "batch seq hidden -> batch seq", "sum")
```

reduce 可以求和、均值、最大值等；输出中消失的轴被归约。`sum` 与 `mean` 的缩放不同，会影响损失与梯度的大小，不能只检查输出形状。

3.3 rearrange：拆轴、换轴、合轴，不等于做乘加

沿用讲义 $(3, 8)$ 的输入，把总宽度 $8 = 2 \times 4$ 拆成两个头，每头作用同一个 $(4, 4)$ 权重，再合回去。

```
x = rearrange(x, "seq (heads din) -> seq heads din", heads=2)
x = einsum(x, w, "seq heads din, din dout -> seq heads dout")
x = rearrange(x, "seq heads dout -> seq (heads dout)")
```

这里两个头**共享同一个** W。若每头有独立参数，权重应多一个 heads 轴，参数量和写法都会改变。rearrange 保持元素总数；有时能返回视图，有时需要复制，不能一概当作零显存、零耗时。

> **读一段张量代码时，先写输入形状，再标出求和轴，最后写输出形状。形状一旦明确，显存与 FLOPs 都有了起点。**

4 核心例子：从一次矩阵乘法推到整次训练

本节目标：推导 $2Bdk$、反向的两次矩阵乘法，以及 $6NT$ 近似的适用范围。

4.1 FLOPs 是工作量，FLOP/s 才是速度

一次浮点加法或乘法通常各计 1 FLOP；融合乘加虽可由一条硬件指令执行，惯例仍计 2 FLOPs。FLOPs 表示总工作量，FLOP/s 表示每秒工作量，TFLOP/s 中的 T 表示 10^{12}。

对 $Y = XW$，$X : (B, d)$、$W : (d, k)$、$Y : (B, k)$，每个输出需 d 次乘法与 $d-1$ 次加法，所以

$$F_{\text{exact}} = Bk(2d-1) \approx 2Bdk.$$

两个同形矩阵相加则只需 Bk 次加法。大尺寸时矩阵乘法通常主导 FLOPs；这句话尚未说明谁主导**耗时**，后面还需计入搬运。

回到小网络：$B = 4, d = 8, L = 3, N = 192$，每层前向约 $2 \times 4 \times 8^2 = 512$ FLOPs，三层约 1536 FLOPs。若严格只数乘加，则为 $3 \times 4 \times 8 \times 15 = 1440$；ReLU 与损失另计。**精确计数与量级估算不能混成同一个数字**。

4.2 先用一个标量损失理解 backward 在求什么

官方例子为 $x = (1, 2, 3)$、$w = (1, 1, 1)$，预测 $p = x^\top w = 6$，目标为 5：

$$\mathcal{L} = \tfrac{1}{2}(p-5)^2 = \tfrac{1}{2}, \qquad \nabla_w \mathcal{L} = (p-5)x = (1, 2, 3).$$

```
w = torch.tensor([1., 1., 1.], requires_grad=True)
loss = 0.5 * (x @ w - 5).pow(2)
loss.backward()   # w.grad == tensor([1., 2., 3.])
```

`requires_grad=True` 告诉自动微分系统跟踪相关运算。默认主要在需要梯度的叶子张量上保留 `.grad`；中间量通常不会保留可直接读取的梯度，可用 `retain_grad()` 调试。`.grad` 为 None 不等于这一步没有参与反向计算。

4.3 为什么一个线性层的反向约为前向的两倍？

设上游梯度 $G = \partial \mathcal{L}/\partial Y \in \mathbb{R}^{B \times k}$，则

$$\frac{\partial \mathcal{L}}{\partial X} = GW^\top \quad (B \times d), \qquad \frac{\partial \mathcal{L}}{\partial W} = X^\top G \quad (d \times k).$$

两次矩阵乘法各约 $2Bdk$ FLOPs，合计约 $4Bdk$。反向既要把梯度传给前层，又要求当前参数的梯度；后一式还解释了为什么反向常需要保留前向输入 X。

4.4 从每层相加，得到 $6N$ 这条估算规则

等宽网络有 $N = Ld^2$ 个参数。忽略激活函数、损失、优化器等较小项，并按各层都求输入与参数梯度估算：

$$F_{\text{forward}} \approx 2BN, \qquad F_{\text{backward}} \approx 4BN, \qquad F_{\text{train step}} \approx 6BN.$$

小网络得到 $6 \times 4 \times 192 = 4608$ FLOPs。这是统一口径的**粗估**；若首层输入不需要梯度，实际会省去相应矩阵乘法。检查点重算还会额外增加工作量。

语言模型中，每条序列有多个 token。将 T 定义为**整次训练累计处理的 token 数**，则常用

$$\boxed{F_{\text{training}} \approx 6NT.}$$

这里 T 不是单条序列长度，N 也应与所计入的主要稠密参数运算匹配。该近似适合矩阵投影等项占主导的情形；长上下文注意力、词表投影、稀疏激活的 MoE、检查点重算等，需要另外核算，不能无条件套用全部参数量。

4.5 回到开场问题：70B 模型训练 15T token 要多久？

沿用讲义 H100 SXM 的估算口径：稠密 bf16 Tensor Core 峰值 $P = 1979/2 = 989.5\,\text{TFLOP/s}$，1024 张卡，假设有效利用率 $u = 0.5$。

$$F = 6 \times 70 \times 10^9 \times 15 \times 10^{12} = 6.3 \times 10^{24}\ \text{FLOPs},$$

$$t \approx \frac{F}{1024 \times 989.5 \times 10^{12} \times 0.5} \approx 1.244 \times 10^7\ \text{s} \approx 143.9\ \text{天}.$$

这不是工期承诺：假设利用率已能概括主要运行损耗，没有另算停机、数据处理、评估等时间；换硬件或计算类型时，要换对应峰值。稀疏峰值不能直接用于普通稠密矩阵乘法。

4.6 MFU：离所选硬件峰值还有多远？

若时间窗口内模型的有效计算量为 F_{model}，测得耗时为 t，则

$$\text{MFU} = \frac{F_{\text{model}}/t}{n_{\text{GPU}} P_{\text{peak}}}.$$

分子分母必须使用一致的精度、稠密 / 稀疏和统计范围。重算是实际执行的额外工作，一般不当作模型有效 FLOPs 加进 MFU 分子。小网络也不能期待接近大矩阵的峰值。

计时补充：CUDA 执行通常异步。应预热、重复测量，并在计时边界同步，或使用 CUDA events；只计 Python 提交运算的时间会低估耗时。算完 FLOPs 后，还要解释实际吞吐为什么达不到峰值，这就是下一节。

5 理解速度：为什么算得少，也可能被显存带宽卡住？

本节目标：把 FLOPs 与搬运字节数放在一起，用 roofline 判断瓶颈。

5.1 一次运算有两项成本：计算与搬运

即使数据已经在 GPU 上，仍需从显存 HBM 读取，送到计算单元，最后写回结果。本节讨论的是 **GPU 内部显存与计算单元之间的搬运**，不是把 CPU 到 GPU 的拷贝混进来。

设运算量为 F、HBM 搬运量为 Q byte、计算峰值为 P FLOP/s、显存带宽为 β byte/s，则

$$t_{\text{compute}} = \frac{F}{P}, \qquad t_{\text{memory}} = \frac{Q}{\beta}, \qquad t \gtrsim \max\left(\frac{F}{P}, \frac{Q}{\beta}\right).$$

右边是理想条件下的**耗时下界**。讲义假设计算与搬运充分重叠，用两者较大项近似时间；真实运行还受启动、同步、缓存、调度等因素影响。

5.2 算术强度：每搬 1 byte，能做多少计算？

定义工作负载的算术强度 I，以及硬件的算力 / 带宽比 I_*：

$$I = \frac{F}{Q}, \qquad I_* = \frac{P}{\beta}, \qquad \text{单位均为 FLOPs/byte.}$$

若 $I < I_*$，搬运相对更慢，称为 memory-bound（带宽受限）；若 $I > I_*$，计算相对更慢，称为 compute-bound（计算受限）。注意，这里的 memory-bound **不是显存容量不足**。

沿讲义的 H100 数值 $P = 989.5 \times 10^{12}$、$\beta = 3.35 \times 10^{12}$：

$$I_* \approx 295.4 \text{ FLOPs/byte.}$$

这个阈值对应选定的稠密 bf16 计算峰值；逐元素运算未必在相同计算单元上执行，其细致分析要用相应运算的峰值。这里按课堂简化模型建立量级直觉。

5.3 roofline：斜线是带宽天花板，横线是计算天花板

由时间下界可推出有效吞吐上界：

$$P_{\text{achievable}} \le \min(P, \beta I), \qquad \frac{P_{\text{achievable}}}{P} \le \min\left(1, \frac{I}{I_*}\right).$$

横轴是算术强度、纵轴是吞吐：左侧随 I 增大而上升，右侧达到计算天花板，拐点为 I_*。真实 MFU 还会更低，不能把这个**理想上界**直接当作实测值。

> **带宽受限时，优先减少搬运或提高数据复用；计算受限时，再考虑减少计算或提高矩阵乘法效率。优化前，先判断处在哪一侧。**

5.4 五个例子：为什么大矩阵乘法特别适合 GPU？

沿用讲义的 bf16 假设：每个输入 / 输出元素 2 byte，输入从 HBM 读一次、输出写一次，暂不计额外中间缓冲。ReLU 比较按一次基本操作计；GELU 的 20 次操作只是课堂粗估。

运算	工作量 F	搬运量 Q	强度 I
ReLU，n 个数	n	$4n$	$1/4$
GELU，n 个数	约 $20n$	$4n$	约 5
长度 n 的内积	$2n-1$	$4n+2$	趋近 $1/2$
$n \times n$ 矩阵乘向量	$n(2n-1)$	$2n^2+4n$	趋近 1
两个 $n \times n$ 矩阵相乘	$n^2(2n-1)$	$6n^2$	约 $n/3$

前三类到矩阵向量乘法，强度不会随规模显著增长；大矩阵乘法则能反复复用每个输入元素。$n = 1024$ 时，矩阵乘法强度约 $341.2 > 295.4$，在此理想模型中进入计算受限区。

对于 $n = 2^{20}$ 的独立 ReLU，搬运 4 194 304 byte，理想搬运时间约 1.25 μs，按课堂峰值估的计算时间约 1.06 ns。两者相差很大；实际计时还会有启动等成本。GELU 虽多做运算，也可能主要等同样的数据搬运，因此减少 FLOPs 未必带来等比例加速，**但不能据此保证二者实测一样快**。

5.5 用同一个公式连接训练、批处理和生成

对 bf16 的 $(B, d) \times (d, k)$ 矩阵乘法：

$$I \approx \frac{2Bdk}{2(Bd + dk + Bk)} = \frac{Bdk}{Bd + dk + Bk}.$$

当 B 相对 d, k 很小时，读权重 dk 占主导，$I \approx B$。单条序列逐 token 解码常接近矩阵向量乘法，读一遍权重只服务很少 token；把更多 token 合批，能摊薄权重搬运，增加复用。

训练中的投影通常同时处理多个 token，矩阵更大，更容易利用算力。这里不能概括为“所有训练都计算受限，所有推理都带宽受限”：预填充、大批量解码、注意力和 KV cache 都会改变账本。

5.6 这也解释了为什么融合运算有用

补充解释：若矩阵乘法后紧接激活，分开执行可能把中间结果先写回 HBM，再读回来。融合可减少这次往返。数学结果和主要 FLOPs 近似不变，但 Q 下降，速度仍可能提升。更深入的 kernel 与 GPU 优化留待后续课。

6 把账算完整：训练状态、优化器与两种省显存方法

本节目标：不漏算梯度和优化器状态；理解梯度累积与激活检查点分别减少什么。

6.1 从一个训练循环理解张量的生命周期

```
optimizer.zero_grad(set_to_none=True)
for x, target in batches:
    pred = model(x)
    loss = criterion(pred, target)  # a scalar loss
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

前向保留反向需要的信息；backward 累加参数梯度；step 用梯度和状态更新参数；zero_grad 清理梯度。**step 通常不自动清梯度**。set_to_none 设梯度为 None，分配器仍可能保留缓存，驱动看到的占用未必立即下降。

官方网络用 `nn.Parameter` 注册权重，用 `nn.ModuleList` 注册多层模块，使参数遍历、设备移动和状态保存能找到它们。普通输入与中间激活不会仅因是张量就自动成为可训练参数。

6.2 优化器为什么还要保存一份“历史”？

AdaGrad 对每个参数维护累计梯度平方。按讲义实现的稳定项位置，逐元素更新为

$$v_t = v_{t-1} + g_t^2, \qquad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t + \varepsilon}}.$$

经常累积较大梯度的坐标，后续更新会被缩小。v_t 与参数形状相同，因此即使参数不变多，也多出一份状态。AdaGrad 是**累加**平方，不要与 RMSProp 的指数滑动平均混淆。

动量法保留平滑梯度，RMSProp 保留梯度平方的指数平均，Adam 同时保存一阶与二阶统计；AdamW 增加解耦权重衰减。本课重在状态成本，不展开完整配方。

6.3 小网络的显存账：按用途逐行相加

假设参数和梯度为 bf16，优化器状态为 fp32；用 $N = 192$、$B = 4, d = 8, L = 3$。按课堂的简化激活模型估算：

项目	计算式	字节数
参数	$2N$	384
梯度	$2N$	384
AdaGrad 状态	$4N$	768
激活（粗估）	$2BLd$	192
合计（AdaGrad）	$8N + 2BLd$	1728
合计（Adam）	$12N + 2BLd$	2496

6.4 从小网络推广：每参数 12 byte 是带前提的

上一页 Adam 的 $12N$ 来自：2 byte 参数、2 byte 梯度、两份各 4 byte 的统计量。它**尚未包含**激活、fp32 主权重副本、输入数据、临时空间与分配器开销。激活的 $2BLd$ 也只是量级模型，不是 PyTorch 自动微分保存所有张量的精确峰值。

开场的 8 张 80 GB H100，在理想完整切分模型状态的条件下，仅按 $12N$ 算：

$$N_{\max} \lesssim \frac{8 \times 80 \times 10^9}{12} \approx 53.3 \times 10^9 \quad \text{个参数.}$$

这是**只算模型状态的理想容量上界**。若普通数据并行在每卡复制完整状态，就不能相加使用 8 张卡显存；单卡相同口径上界约为 6.67B。若另有 fp32 主权重，状态变成 $16N$，8 卡聚合上界便降到 40B，激活等还需继续扣除。

6.5 梯度累积：把一个大批次拆成多个小批次

设有效批大小为 B，拆为 K 个相等的 micro-batch，每个大小 $b = B/K$。逐个做前向、反向，累积梯度，全部完成后只更新一次。这样可及时释放前一个 micro-batch 的计算图，激活峰值从约 $2BLd$ 变为 $2bLd$；参数、梯度和优化器状态并未因此缩小。

```
# K equal-sized micro-batches; criterion returns their mean loss.
optimizer.zero_grad(set_to_none=True)
for x, target in micro_batches:
    loss = criterion(model(x), target) / K
    loss.backward()
optimizer.step()
optimizer.zero_grad(set_to_none=True)
```

若每个 micro-batch 损失已取均值，需要除以 K 才与大批次均值对应；大小不等时按样本数加权，语言模型按参与损失的有效 token 数加权。不能在每个 micro-batch 后清梯度或更新参数，否则变成另一种训练过程。

讲义例子 $B = 64, d = 1024, L = 16$：简化 bf16 激活量为 2 MiB；拆成 4 个大小 16 的 micro-batch 后约为 0.5 MiB。总样本工作量近似不变，但小矩阵的算术强度和硬件效率可能下降。

> **梯度累积解决“同一次更新想看更多数据，但激活放不下”的问题。它通常不减少模型状态，也不承诺提升速度。**

等价性前提：损失可按样本 / token 分解且参数在累积期间固定；批统计、随机性及浮点求和次序可能带来差异。梯度裁剪等操作通常应在累积完成后执行。

6.6 激活检查点：少存一些，需要时重新算出来

反向需要前向中间量，例如 $\partial\mathcal{L}/\partial W = X^\top G$ 中的 X。常规方式把这些信息保存到反向；activation checkpointing（也叫 gradient checkpointing 或 rematerialization）则只保留部分边界，反向时从最近边界重做前向，恢复缺失激活。

它保存的是**反向所需的激活检查点**，不是训练中保存到磁盘、用于恢复任务的模型 checkpoint。两者解决的问题不同。

对各层激活大小相同的 L 层链，设每隔 k 层保存一个边界。反向处理一段时，既需要所有边界，也需要暂存当前段的中间量，因此简化峰值为

$$M_{\text{activation}} = O\left(Bd\left(\frac{L}{k}+k\right)\right), \qquad k \approx \sqrt{L} \implies O(Bd\sqrt{L}).$$

字节数还需乘 dtype 宽度。以 $L = 16$ 为例，每 4 层一个边界，避免同时保留全部层内部激活；这给出的是量级关系，不是精确节省比例。

策略（简化链模型）	激活空间量级	额外重算工作量
保存所有层所需激活	$O(L)$	无需重算前向。
逐层从头恢复、只留常数状态	$O(1)$	总重算可达 $O(L^2)$。
每约 $\sqrt{L}$ 层保存边界	$O(\sqrt{L})$	总额外重算为 $O(L)$。

表中省略了 Bd 与 dtype 等共同因子。最后一行的 $O(L)$ 指**所有段加起来的重算**；不是把段长 $O(\sqrt{L})$ 当作全网重算量。实际选择取决于模块大小、可用显存和重算开销。

PyTorch 接口补充：可用 `torch.utils.checkpoint.checkpoint` 包裹需要重算的函数；按当前文档应明确选择 `use_reentrant=False`。被包裹函数的重算需与原前向一致；随机状态、副作用或可变全局状态要遵守接口约定。课堂“每层包一次”的例子和“每隔 $\sqrt{L}$ 层分段”的理论策略也不是同一种分段方式。

6.7 两种方法怎样共同服务主线？

梯度累积缩小**同时处理的数据量**；激活检查点减少**同时保留的中间结果**。前者通常维持近似总 FLOPs，后者明确增加计算。它们可组合使用，但都不消除模型状态成本。

优化闭环：先列出显存最大项，再选方法；改完重新计数 FLOPs 与搬运，并用实际峰值和计时验证。能放下、算得少、跑得快，是三个相关但不同的目标。

7 一页速查：把本课压回一条主线

快速复习时先读这一页；无法解释某条结论时，回到对应章节看推导。

7.1 用六句话重建本课

1. 文本变成张量之后，要同时核算显存容量、总运算量与数据搬运，才能判断训练方案是否可行。
2. 张量逻辑数据量等于元素数乘 dtype 字节数；参数、梯度、优化器状态与激活应分别列账。
3. einops 显式写出轴的含义：einsum 做乘积与求和，reduce 做归约，rearrange 做拆合和重排。
4. 线性层前向约 $2Bdk$，反向两次矩阵乘法约 $4Bdk$，从而得到常用训练粗估 $6NT$。
5. 算术强度 $I = F/Q$ 决定算力和带宽谁更紧；大矩阵乘法通过复用输入提高强度。
6. 梯度累积减少 micro-batch 激活峰值，检查点用额外重算换激活空间；两者都有适用条件与代价。

7.2 五条公式与一个连续例子

记忆点	对应含义与边界
$M = (\prod_i s_i)q$	稠密张量逻辑字节数；共享存储与分配开销另计。
$F_{\text{matmul}} \approx 2Bdk$	$(B, d) \times (d, k)$；前向乘加估算。
$F_{\text{train}} \approx 6NT$	N 为对应参数量，T 为累计 token 数；不含额外重算等。
$I = F/Q,\ I_* = P/\beta$	同为 FLOPs/byte；$I < I_*$ 时理想模型带宽受限。
$t \gtrsim \max(F/P, Q/\beta)$	时间下界；吞吐上界为 $\min(P, \beta I)$。
$B = 4, d = 8, L = 3$	$N = 192$；前向约 1536 FLOPs，训练约 4608；简化 AdaGrad 显存 1728 byte。

7.3 易错点：看到这些说法，要能立即纠正

- **“bf16 和 fp32 一样精确。”** 范围接近，不等于有效数字位数相同。
- **“有 8 卡，就能直接相加显存。”** 取决于状态是否切分；复制式数据并行不成立。
- **“低 FLOPs 就一定快。”** 还要看字节搬运、形状和启动等开销。
- **“memory-bound 就是显存不够。”** 此处是带宽受限，容量另算。
- **“累积梯度只要多次 backward。”** 还需正确缩放损失、控制更新与清梯度时机。
- **“检查点免费省显存。”** 激活减少，但多了前向重算；MFU 核算也需统一口径。

8 自测：合上前文，检查自己是否真的理解

建议先独立作答，再看第 9 节。每题都尝试说出计算前提与单位。

8.1 概念题：检查因果关系

Q1. 三本账。同一个模型换上容量更大、但带宽和算力相同的 GPU，为什么可能能训练更大 batch，却不保证单次运算更快？

Q2. 精度与存储。形状 $(32, 16, 16, 64)$ 的张量有多少元素？fp32、bf16 分别占多少 MiB？为什么 bf16 能表示 10^{-8}，却不能据此说它与 fp32 一样精确？

Q3. 形状判断。输入 $X : (2, 3, 4)$、$Y : (2, 5, 4)$，执行 `einsum` 模式 `batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2`，输出形状与求和轴是什么？与沿 hidden 求均值有何不同？

8.2 手算题：检查操作是否准确

Q4. 前向和反向。$X : (4, 8)$、$W : (8, 8)$。用 $2Bdk$ 口径估计前向 FLOPs；写出两项反向矩阵公式及各自形状。如果该层输入不需要梯度，会少做哪一项？

Q5. 完整显存账。三层无偏置、不共享权重的等宽网络，$B = 4, d = 8, L = 3$。参数和梯度用 bf16，AdaGrad 状态用 fp32，激活按 $2BLd$ byte 粗估。参数量与总显存是多少？改成两份 fp32 状态的 Adam 后呢？

Q6. 算术强度。bf16 的 1024×1024 方阵乘法，假设两个输入各读一次、输出写一次。估算 F, Q, I，与 $I_* = 295.4$ 比较。若换成矩阵向量乘法，为什么强度会大幅下降？

8.3 迁移题：检查是否会判断取舍

Q7. 利用率与工期。用 $6NT$ 估计 70B 模型、15T token 的总计算量。1024 张卡、每卡峰值 989.5 TFLOP/s、MFU 为 0.5，要多少天？若 MFU 变为 0.25，其余不变，时间如何变化？

Q8. 梯度累积。有效 batch 为 64，分成 4 个等大的 micro-batch，各自损失都已取均值。应怎样缩放损失，何时 step 和 zero_grad？哪类显存减少，哪类不减少？

Q9. 激活检查点。等宽 L 层链每隔 k 层保留边界，为什么激活峰值含 $L/k + k$ 两项？怎样选 k？为什么“总重算为 $O(\sqrt{L})$”不正确？它能否解决参数本身就装不下的问题？

> **掌握标准：能从形状算字节和 FLOPs，解释反向的两次矩阵乘法，用算术强度判断瓶颈，并根据显存最大项选择优化方法。**

延伸观察：任选一段现有训练代码，列出每类张量的形状、dtype 与生命周期，再对照实测峰值。先解释差值来自哪里，不急于追求一个看似精确的总数。

9 参考答案与资料来源

答案用于核对关键点，不要求逐字背诵；数值均沿正文给定的简化口径。

9.1 参考答案

A1. 容量增大允许保存更多参数、状态或激活，但没有提高每秒计算或搬运速度。大 batch 可能提高复用和吞吐，也会增加每步工作量，所以单次运算或每步耗时不能由容量直接推出。

A2. 元素数为 $32 \times 16 \times 16 \times 64 = 524\,288$；fp32 为 2 097 152 byte，即 2 MiB；bf16 为 1 MiB。bf16 与 fp32 都有 8 位指数，但 bf16 尾数仅 7 位，fp32 有 23 位；范围和精度是不同属性。

A3. 输出 $(2, 3, 5)$；hidden 轴上的对应元素相乘后求和，保留 batch 与两个位置轴。沿 hidden 求均值是单张量的归约，不产生两两位置的内积矩阵，也有不同的缩放。

A4. 前向约 $2 \times 4 \times 8 \times 8 = 512$ FLOPs。若上游 $G : (4, 8)$，则 $GW^\top : (4, 8)$，$X^\top G : (8, 8)$，各约 512 FLOPs，反向合计约 1024。不需要输入梯度时，可省去 $GW^\top$，但仍要求权重梯度。

A5. $N = 3 \times 8^2 = 192$。参数 384、梯度 384、AdaGrad 状态 768、激活粗估 192 byte，总计 1728 byte。Adam 两份状态合计 1536 byte，总计 2496 byte；均未计输入、额外中间量与运行开销。

A6. 近似 $F = 2 \times 1024^3 = 2\,147\,483\,648$ FLOPs，$Q = 6 \times 1024^2 = 6\,291\,456$ byte，$I \approx 341.3$；若精确计点积加法则 $I \approx 341.2$，均大于 295.4，在理想模型中计算受限。矩阵向量乘法约 $2n^2$ FLOPs，却仍需读约 $2n^2$ byte 权重，强度趋近 1，权重复用少。

A7. 总计算量约 6.3×10^{24} FLOPs。时间为该数除以 $1024 \times 989.5 \times 10^{12} \times 0.5$，再除以 86400，约 143.9 天。MFU 减半则估计时间翻倍到约 287.9 天；这一结论以其他条件固定为前提。

A8. 每个大小 16 的 micro-batch 均值损失除以 4，再 backward；四次全部完成后只 step 一次，并清梯度。简化激活峰值约降至四分之一；参数、梯度、优化器状态基本不变。不等长或有 mask 时要按实际参与损失的样本 / token 加权。

A9. L/k 项用于保存各段边界，k 项用于反向时恢复并保存当前段中间量；平衡得 $k \approx \sqrt{L}$，激活量级为 $O(\sqrt{L})$。每段重算约 k 层，约 L/k 段，总额外重算为 $O(L)$。它主要节省激活，不能消除参数本身的占用。

9.2 阅读范围与来源

核心依据为 **Spring 2026 第 2 课正式可执行讲义、官网标注的录像版可执行讲义及其代码和执行记录**。本资料按问题链重新组织，不是录像逐字稿；没有把读取录像版讲义描述为观看完整视频，也不声称逐篇精读所有背景论文。

- 课程主页与 Schedule：第 2 课日期为 2026 年 4 月 1 日，主题为 PyTorch（einops）与资源核算。
- 正式讲义源码；正式在线讲义；录像版在线讲义：张量、精度、FLOPs、roofline、梯度、优化器及显存优化的全部主体。
- einops 官方教程：轴的组合、拆分、重排与归约。
- PyTorch AMP 文档；checkpoint 文档：补充核对 autocast 的适用范围与检查点接口约定。
- Mixed Precision Training：fp32 主权重、loss scaling 的背景；FP8 Formats for Deep Learning：E4M3 与 E5M2；NVIDIA NVFP4 说明：低位宽及缩放开销。只提取本课相关机制，不展开完整训练配方。
- AdaGrad 原论文：核对累计平方梯度与逐坐标调整的含义，不展开收敛证明。
- All About Rooflines：核对时间界、强度与数据复用；NVIDIA H100 官方资料及课程 facts.py：硬件口径。正文保留课堂采用的数值，不把它当作所有 H100 型号与精度的统一规格。
- 讲义延伸入口：Transformer FLOPs（已读取，对照 $6N$ 与 MFU 的边界）；Transformer 显存分析（本次返回 404，未用其不可访问内容作依据）。

9.3 版本差异与补充解释的边界

正式讲义、部署的执行快照和录像版并非完全相同。录像版开场写 B100，代码却使用 H100 峰值；另有 “2 周” 标题但只乘 1 周、把参数字节数再次乘字节宽度、将梯度累积元素数称为 memory，以及检查点总重算复杂度等不一致。正文按**单位一致**、**公式可推导**的口径核对，不复制这些数值或文字错误。

激活字节数与 PyTorch 实际分配、AMP 与永久存储类型的区别、8 卡聚合的切分前提、MFU 的理想界、损失加权与检查点分段推导，是为复习补充的澄清。课堂训练循环只演示接口；正文用标准标量损失流程说明，不把演示数据或广播行为当作真实任务配置。

讲义还列出 Transformer 显存和 FLOPs 的延伸文章，以及模型案例、训练动态和课程行政链接。它们属于背景或后续入口；本资料不将模型架构、历史排行榜、未公开模型的估算或作业答案扩展为本课要求。所有核心章节已按正式与录像版逐项覆盖。

公开资料可能继续更新；整理日期为 2026 年 10 月 23 日。源码版本和执行快照指纹、数值及排版检查记录见同目录 README 与检查记录；原始资料通过以上官方链接访问。